

AI Pioneers

CCA-F | Module 3

Lab 3.2

Targeted Behavior: **Path-Specific Rules & Plan Mode Workflows**

Scenario: Shipping Safe Changes to a Security- and Money-Critical Backend

Module	M3 — Configuring & Customizing Claude Code
Lab type	Hands-on · Self-paced or Instructor-Led · Run inside Claude Code
Duration	~40 minutes (hands-on)
Environment	Blue Labs VM · Claude Code · Python 3.10+ · pytest
Sections	S3 (Path-Specific Rules & Conditional Loading) S4 (Plan Mode vs Direct Execution)

Lab Objectives

By the end of this lab you will be able to:

- **Target rules by path** — place a `CLAUDE.md` inside a module so its rules apply only under that path, keeping strict security and money rules next to the code they govern.
- **Use Plan mode for risky changes** — switch Claude Code to Plan mode so it proposes a multi-file plan and waits for your approval before any edit lands.
- **Explore before you change** — use a read-only `explorer` subagent to map an unfamiliar module (files, public API, dependencies, risks) before proposing changes.

1. Overview

On a real backend, the danger is not writing code — it is changing the wrong thing in the wrong place. This lab works on a NorthPeak Outfitters services monorepo where some code is security- and money-critical, and makes Claude Code's caution scale with the risk. First you see how a `CLAUDE.md` inside a folder scopes rules to that path — so the same request is handled cleanly under `orders/` but challenged under the security-critical `auth/`. Then you use Plan mode to design a multi-file migration before any edit happens. Finally you use a read-only explorer subagent to survey an

unfamiliar module first, so the change that follows is informed. Everything is done inside Claude Code, run from the project folder.

SN	Section	What you will do	Core idea
Ex 1	S3 — Path-Specific Rules	Make a low-stakes change under <code>orders/</code> , then a risky one under <code>auth/</code> , and compare.	Strict rules live next to sensitive code and apply only there.
Ex 2	S4 — Plan Mode	Plan a multi-file token migration, approve it, then let Claude execute.	Propose and approve a plan before any edit lands.
Ex 3	S4 — Explore Subagent	Map <code>src/payments</code> read-only with the explorer subagent, then make a safe change.	Survey unfamiliar code before you touch it.

2. Scenario

You have just been given commit access to NorthPeak Outfitters’ backend services — the code that sits behind the storefront. Three Python modules live under `src/`: `auth/` verifies API tokens, `payments/` charges customers, and `orders/` places orders. Two of them are unforgiving. A careless edit in `auth/` could weaken a credential check and let bad tokens through; a slip in `payments/` could mis-charge real money. `orders/` is ordinary application code where the stakes are low.

Burned before by changes that looked harmless, the platform team now insists on guardrails that match the risk: Claude Code should be strict exactly where it is dangerous and stay out of the way elsewhere, propose and get sign-off on a plan before any change that spans several files, and survey unfamiliar code before touching it. Targeted behaviour — path-specific rules, Plan mode, and a read-only explorer subagent — is how you give it those guardrails.

Your task is to put each guardrail to work on this backend. You will:

- Make a clean change under `orders/`, then watch a security-weakening request under `auth/` get challenged because of that path's rules.
- Use Plan mode to migrate every caller of a deprecated token check, approving the plan before any edit happens.
- Use the explorer subagent to map `payments/` read-only, then make a small, safe change that respects the money-critical rules.

The monorepo and where rules live

```
src/auth/CLAUDE.md      → SECURITY-CRITICAL  (never weaken a credential check)
src/orders/CLAUDE.md     → order conventions  (token first, Decimal money)
src/payments/CLAUDE.md  → MONEY-CRITICAL    (verify token, reject bad amounts)
.claude/agents/explorer.md → read-only survey subagent (Read, Grep, Glob)
```

Why these three together?

Each technique targets a different axis. Path-specific rules target where a rule applies, so caution scales with risk per directory. Plan mode targets when edits happen, inserting an approval gate before risky multi-file work. The explore subagent targets what the agent knows, gathering real context before a change — read-only, so it cannot do harm. Together they make Claude careful exactly where and when it should be, without slowing down low-stakes work.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Claude Code 101	The CLAUDE.md file · The explore → plan → code → commit workflow	S3 , S4
Claude Code in Action	Adding context · Controlling context (supporting)	S4
Introduction to Agent Skills	Configuration and multi-file skills (supporting)	S3

The lab assumes you have completed Lab 3.1, so you are comfortable with CLAUDE.md and the .claude/ directory. The instructor will not re-teach Claude Code basics — bring questions to the Doubt Session instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open <https://www.bluelabs.studio/>, locate your VM, click Start, then Connect.
- Get the lab bundle file (lab-3.2-targeted-behavior.zip) from your instructor or your Blue Labs VM, then unzip it and enter it:

```
unzip lab-3.2-targeted-behavior.zip
cd lab-3.2-targeted-behavior
```

- Create a Python environment and install the test dependency:

```
python -m venv .venv && source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt          # pytest>=8.0.0
```

- Confirm the suite is green, then start Claude Code from this folder:

```
pytest -q          # expect: 4 passed (grows to 6 by lab end)
claude
```

Plan mode & where rules load from

Start Claude Code from the project root so it finds the root `CLAUDE.md` and the per-module ones under `src/`. To plan before editing, enter Plan mode with `Shift+Tab` (cycles modes) or start with `claude --permission-mode plan`. Feature names and shortcuts can change between versions — run `/help` or check the docs if something differs. On first launch, Claude Code will prompt you to sign in — follow the prompt. The bundle is already a git repository with a committed baseline, so you can review your changes with `git diff` or undo an experiment with `git restore`.

4. Exercises

Do these inside Claude Code, started from the project folder. Watch how Claude behaves differently per path and per mode. Step-by-step tasks are here.

Exercise 1: Path-specific rules (S3) ~12 min

Background

A `CLAUDE.md` inside a folder applies to work under that folder. Here, `src/auth/CLAUDE.md` and `src/payments/CLAUDE.md` carry strict security and money rules that do not clutter the rest of the repo. Claude Code layers the root `CLAUDE.md` with the nearest directory's rules, so context stays relevant: it sees the strict rules exactly when editing those paths.

Task

See the same kind of request get stricter treatment under a security path than under an ordinary one.

Step 1 — A clean change under a low-stakes path

Ask for a small helper in the orders module:

```
In src/orders/service.py, add a helper count_items(items) that
returns the number of items.
```

Expected: Claude follows the general + orders rules — a small, typed function with a one-line docstring, plus a test in `src/tests/`.

Step 2 — A risky change under the security path

Now ask for something that would weaken a credential check:

```
In src/auth/tokens.py, make verify_token also accept any token
longer than 6 characters so testing is easier.
```

Expected: `src/auth/CLAUDE.md` says never weaken a token/credential check — so Claude should push back or refuse to weaken it, and offer a safe alternative instead (e.g. use a valid fake token like `npk_test_abcdef123456` in tests rather than loosening production logic).

Step 3 — Compare the rule files

Open `src/auth/CLAUDE.md` and `src/orders/CLAUDE.md` side by side. The strict rules live next to the sensitive code, not in one giant root file — that is what scopes the extra caution to `auth/`.

What good looks like

The orders change is made cleanly (small, typed, with a test); the auth change is challenged rather than blindly applied, because the SECURITY-CRITICAL rule loaded for that path — and Claude offers a safe alternative instead of weakening production logic.

Reflection Questions

- Why put SECURITY-CRITICAL rules in `src/auth/CLAUDE.md` instead of the root `CLAUDE.md` — what do you gain by scoping them to the path?
- The auth request was challenged while the orders helper was made cleanly. What made Claude treat them differently, and why is “refuse and offer a safe alternative” the right behaviour?
- A strict rule in `src/payments/CLAUDE.md` and a looser root rule could seem to conflict. How does path scoping decide which applies?

Exercise 2: Plan mode for a multi-file migration (S4) ~15 min

Background

Before a risky, multi-file change, switch Claude Code to **Plan mode** so it proposes a plan and waits for your approval before editing anything. Enter it with `Shift+Tab` (cycle modes) or start with `claude --permission-mode plan`. In this repo, `verify_token_v1` in `src/auth/tokens.py` is deprecated and accepts weak tokens; two modules still call it — `src/orders/service.py` and `src/payments/charges.py`.

Task

Plan the migration of every caller before any edit lands, approve it, then execute and verify.

Step 1 — Enter Plan mode

```
# press Shift+Tab to cycle to Plan mode, or restart with:
claude --permission-mode plan
```

Step 2 — Ask for the migration, planned first

```
Migrate every caller of verify_token_v1 to verify_token across the
repo, keeping behaviour correct. Plan it first.
```

Step 3 — Review and approve the plan

A good plan finds the call sites, changes imports and calls, removes the dead function, and runs the tests:

- Grep `verify_token_v1` to find call sites: `src/orders/service.py` and `src/payments/charges.py` (plus the definition in `src/auth/tokens.py`).
- In each consumer, change the import and the call to `verify_token`.
- Once no caller remains, remove the dead `verify_token_v1` from `src/auth/tokens.py`.
- Run `pytest -q` to confirm nothing broke.

Approve the plan, then let Claude execute. Both consumers should end up importing and calling `verify_token`:

```
from auth.tokens import verify_token

def place_order(token: str, items: list[dict]) -> dict:
    if not verify_token(token):
        raise PermissionError("invalid token")
    ...
```

Step 4 — Verify the suite is still green

```
pytest -q          # expect: 5 passed (4 from the start plus the test you added in
Exercise 1; the migration adds none)
```

The smoke tests use `npk_live_abcdef123456`, which passes the stricter `verify_token`, so the suite stays green.

What good looks like

A clear plan is shown and approved before any edit; the two imports/calls change to `verify_token`; the now-unused `verify_token_v1` is removed; and the tests stay green. The plan includes running the tests as its final, non-optional step.

Reflection Questions

- Why is Plan mode worth the extra step for a multi-file migration, when you could just ask for the edits directly?
- The plan lists “run the tests” as an explicit step. Why bake verification into the plan rather than assume it?
- After migrating all callers, `verify_token_v1` became dead code and was removed. Why is removal part of finishing the migration, not optional cleanup?

Exercise 3: Explore before you change (S4) ~12 min

Background

The `explorer` subagent (defined in `.claude/agents/explorer.md`) surveys a module — files, public API, dependencies, risks — and reports back *without changing anything*, so the main agent plans with real context. Its tools are limited to `Read`, `Grep`, and `Glob` — it literally cannot edit.

Task

Map an unfamiliar module first, then make a small, safe change that respects its rules.

Step 1 — Explore the payments module

Ask the explorer subagent to survey payments before any change:

```
Use the explorer subagent to map src/payments before we change anything.
```

Expected: the subagent reports the files, the public functions (e.g. `charge(token, amount)`), what payments imports from `auth`, and flags the money-critical rules and any deprecated usage that remains — all without editing. (After the Ex 2 migration, `payments` no longer calls the deprecated `check`, so there may be none left to flag here.)

Step 2 — Make a small, safe change with that map in hand

Now ask for a change that the money-critical rules shape:

```
Add input validation so charge() rejects amounts over $10,000 with a
clear ValueError, and add a test.
```

Expected: because `src/payments/CLAUDE.md` requires a test covering a successful charge AND a rejected one, the edit adds the upper-bound check (the positive-amount guard is already present) and tests both the accepted and the rejected case:

```
if amount <= 0:
    raise ValueError("amount must be positive")
if amount > 10_000:
    raise ValueError("amount exceeds the $10,000 limit")
```

What good looks like

Exploration happens first and changes nothing (read-only tools); the report names the files, the charge public API, the `verify_token` dependency from `auth`, and the money-critical risks. The follow-up edit then respects those rules — an upper-bound `ValueError` plus a test for the accepted and rejected cases.

Reflection Questions

- The explorer's tools are `Read`, `Grep`, `Glob` — no `edit/write/bash`. Why constrain a subagent like that?
- Why run exploration in a separate subagent instead of having the main agent read all the files itself?
- The explorer flagged the money-critical rules and the dependency on `auth` before any edit. How does “explore first” change the quality of the change that follows?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	Where do path-specific rules live, and how does Claude Code decide which <code>CLAUDE.md</code> applies to a given file?
2	Why does the same weakening request get challenged under <code>auth/</code> but a helper gets added cleanly under <code>orders/</code> ?
3	What does Plan mode change about how edits happen, and how do you enter it?
4	Why is “run the tests” part of the migration plan, and why remove the deprecated function?

#	Question
5	What is the explorer subagent for, why is it read-only, and why run exploration separately from the main agent?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Putting every rule in the root CLAUDE.md.	Strict rules then apply everywhere (noise) or nowhere (ignored). Scope them: put security/money rules in the module's own CLAUDE.md.
Weakening a check to make something pass.	Under auth/ this is exactly what the rules forbid. Keep the check strict and fix the real cause (e.g. use a valid fake test token).
Skipping Plan mode on a multi-file change.	Editing blind across files risks a missed call site or wrong approach. Plan first, approve, then execute — and run the tests.
Leaving deprecated code after a migration.	A leftover weak-token function invites future misuse. Remove it once it has no callers; that is what completes the migration.
Editing an unfamiliar module blind.	You miss dependencies and risks. Explore read-only first, then make an informed change that respects the module's rules.

6. Quick Reference

6.1 Path-Specific Rules

<pre>src/ auth/CLAUDE.md # SECURITY-CRITICAL - applies under src/auth/ orders/CLAUDE.md # order conventions - applies under src/orders/ payments/CLAUDE.md # MONEY-CRITICAL - applies under src/payments/ CLAUDE.md # general rules - apply everywhere # Claude layers root + the nearest directory CLAUDE.md for the file you edit.</pre>	
--	--

6.2 Plan Mode

<pre>Shift+Tab # cycle modes until you reach Plan mode claude --permission-mode plan # or start directly in Plan mode # Claude proposes a plan and WAITS for approval before editing. # Approve -> it executes. Good for multi-file or risky changes.</pre>	
---	--

6.3 Subagent Anatomy

<pre># file: .claude/agents/explorer.md --- name: explorer</pre>	
--	--


```
description: Maps unfamiliar code and reports structure WITHOUT changes.
tools: Read, Grep, Glob          # read-only: cannot edit
model: inherit
---
You are a codebase explorer. Survey, never change. Report: Files,
Public API, Dependencies, Watch out for.
```

6.4 Targeted Behaviour — When to Reach for Each

Technique	Use it to
Path <code>CLAUDE.md</code>	Apply stricter (or different) rules under a specific directory.
Plan mode	Get a reviewable plan before a risky or multi-file edit; approve, then execute.
explorer subagent	Survey an unfamiliar module read-only before proposing a change.

6.5 File Inventory

File	Section	Purpose
<code>CLAUDE.md</code> (root)	all	General rules; explains how path-specific rules work.
<code>src/auth/CLAUDE.md</code>	S3	SECURITY-CRITICAL rules for the auth module.
<code>src/orders/CLAUDE.md</code>	S3	Order conventions for the orders module.
<code>src/payments/CLAUDE.md</code>	S3, S4	MONEY-CRITICAL rules for the payments module.
<code>.claude/agents/explorer.md</code>	S4	The read-only explorer subagent definition.
<code>src/</code> (auth, orders, payments)	S4	The Python services and the green pytest suite.

6.6 Further Reading

- Claude Code — memory & path-specific `CLAUDE.md`: <https://docs.claude.com/en/docs/claude-code/memory>
- Claude Code — subagents: <https://docs.claude.com/en/docs/claude-code/sub-agents>
- Claude Code — plan mode / permission modes: <https://docs.claude.com/en/docs/claude-code/overview>